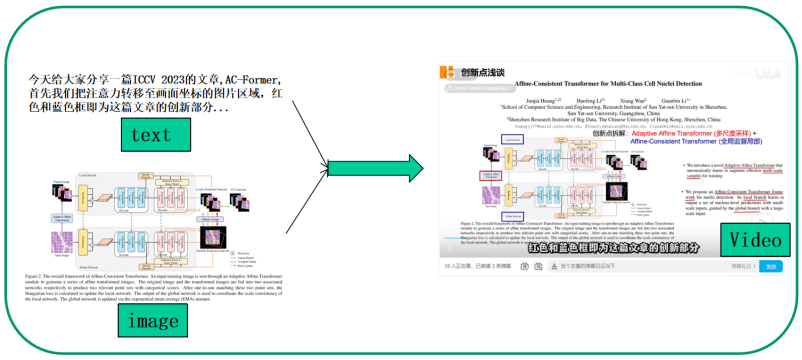
LLM与TTS技术在生成口播类短视频中的应用 - 安宁 & 牛蒙

背景介绍

在**自媒体**时代，大部分人都有分享自己经历或者经验的需求。最初通过公众号或者微博平台，自媒体人员只需要有写作能力就可以获得巨大的流量。而随着短视频的流行，具备将文章变为视频，上传到各类视频平台的能力，会成为自媒体人员的一个优势。但**并不是每个人都具备良好的口播能力**或者视频制作的能力，所以这里提出了一个口播类短视频自动生成应用。

此应用的功能：

1. 使用**大语言模型（large language model，LLM）**对口播文稿进行优化
2. 使用**test-to-speech(TTS)模型**自动生成高质量的**口播音频。**
3. 自动生成**字幕**文件。
4. 自动将**音频文件、字幕文件、图片素材**合并为视频文件。



核心技术

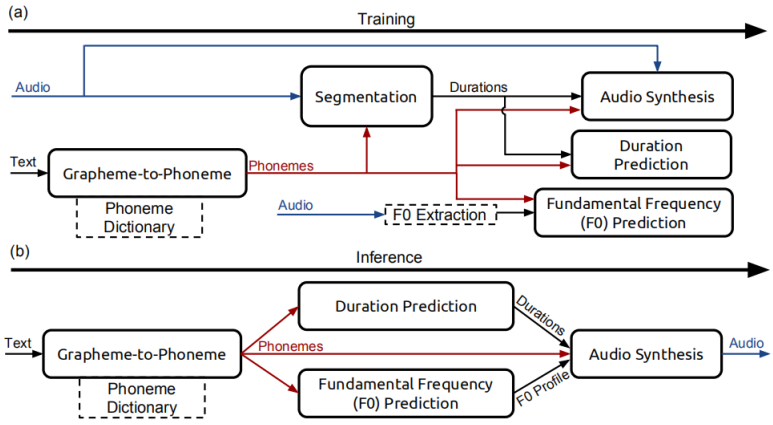
此应用的实现使用到了两项计算机相关的核心技术：**test-to-speech**和**Audio and video processing**.

1. text-to-speech（TTS）

是将计算机自己产生的、或外部输入的**文字信息**转变为可以听得懂的、流利的**口语**输出的技术。



TTS 详细流程**[0]**:

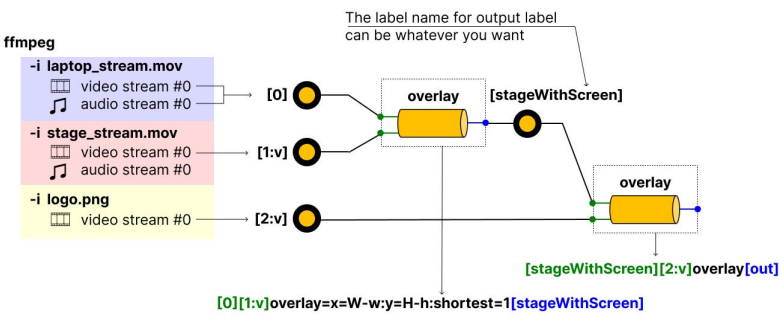


---

2. Audio and video processing

音视频处理本身是需要相当丰富的专业技术来完成。幸运的是FFmpeg框架简化了此过程，并提供一种声明性方式来表达我们处理视频所需的意图[1]。

FFmpeg是处理多媒体内容(如音频、视频、字幕和相关元数据)的库和工具的集合[2]。



参考：

[0] Deep Voice: Real-time Neural Text-to-Speech. 2017. Arik, Sercan O., et al.

[1] How to process video with FFmpeg. Framework syntax from Zero to Hero

[2] https://github.com/FFmpeg/FFmpeg

具体实现

1. 读取文稿文本文件，通过大语言模型对文稿进行优化

这里使用了ChatGPT的API接口，通过角色扮演的方式使ChatGPU扮演一名写作专家。然后让其对口播文稿进行优化。

可以调用大语言模型进行替换。

```
import openai

openai.api_key = "API_KEY"

model_engine = "gpt-3.5-turbo"

input_text = "今天给大家分享一篇ICCV 2023的文章,AC-Former,首先我们把注意力转移至画面坐标的图片区域，红色和蓝色框即为这篇文章的创新部分"

messages=[
        {"role": "system", "content": "You are a helpful writing assistant."},
        {"role": "user", "content": "Please optimize the following document to make it more suitable for oral presentation. {input_text}"},
]

response = openai.ChatCompletion.create(
    model=model_engine,
    messages=[{"role": "user", "content": input_text }]
)

# response将会得到一个类似这样结构的json消息
# {
#   'id': 'chatcmpl-',
#   'object': 'chat.completion',
#   'created': ,
#   'model': 'gpt-3.5-turbo',
#   'usage': {'prompt_tokens': , 'completion_tokens': , 'total_tokens': },
#   'choices': [
#      {
#        'message': {
#            'role': 'assistant',
#            'content': ''},
#        'finish_reason': 'stop',
#        'index': 0
#      }
#    ]
# }

# 解析响应并输出结果
output_text = response['choices'][0]['message']['content']
print("优化后的文稿: {output_text}")

```

2. 读取优化后的文稿文件，通过TTS模型生成音频文件和字幕文件

选择了Microsoft Edge TTS接口，其基于Microsoft Azure服务实现，提供了的语音合成算法,生成的音频文件具有更自然的语音口播效果。此接口通过Python库edge_tts**[3]**实现。

可以通过FaceBook的SeamlessM4T**[4]**，microsoft的SpeechT5**[5]**和NaturalSpeech**[6]**模型进行替换。

参考：

[3] https://github.com/rany2/edge-tts

[4] https://github.com/facebookresearch/seamless_communication

[5] https://github.com/microsoft/SpeechT5

[6] https://speechresearch.github.io/naturalspeech/

```
import asyncio

import edge_tts

TEXT = output_text
VOICE = "zh-CN-XiaoyiNeural"

OUTPUT_FILE = "test.mp3"
WEBVTT_FILE = "test.vtt"


async def _main() -> None:
        communicate = edge_tts.Communicate(TEXT, VOICE)
        submaker = edge_tts.SubMaker()
        with open(OUTPUT_FILE, "wb") as file:
                async for chunk in communicate.stream():
                        if chunk["type"] == "audio":
                                file.write(chunk["data"])
                        elif chunk["type"] == "WordBoundary":
                                submaker.create_sub((chunk["offset"], chunk["duration"]), chunk["text"])

        with open(WEBVTT_FILE, "w", encoding="utf-8") as file:
                file.write(submaker.generate_subs())

await _main()
# if __name__ == "__main__":
        # asyncio.run(_main())


```

+ 代码 + 文本

3. 读取图片素材生成无语音的短视频文件

```
#

cmd = "ffmpeg -f image2 -i %d.jpeg test.mp4"

os.ststem(cmd)
```

4. 将将音频文件、字幕文件、视频文件合成为一个完整的短视频文件

```
#

import os

cmd = "ffmpeg -i test.mp4 -i test.vtt -i test.mp3 -map 0:v -map 2:a -c:v copy -c:a copy -c:s mov_text output.mp4"

os.system(cmd)
```